

DataEntry DSL — User Manual

Introduction

The **DataEntry** tool lets you define text-oriented data-entry screens in a simple language (a `.def` file) and then either:

- **Preview** the form interactively to verify its layout and colours, or
- **Compile & Build** it into a standalone executable that writes fixed-length records to a data file.

The tool itself runs on Windows, Linux, and macOS in console mode. The applications it generates also run on all three platforms.

Package Contents

When you unzip (Windows) or untar (Linux/macOS) a DataEntry release you get:

DataEntry.exe	(Windows) or dataentry (Linux/macOS)
libonigwrap.dll	(Windows) or libonigwrap.so / .dylib - m
INSTALL.md	installation and SDK setup guide
MANUAL.md	language reference and user guide
Samples\ sample.def	simple customer-entry starter form
customer.def	full customer master with address and cont
inventory.def	inventory item entry with validation stubs
timesheet.def	multi-screen weekly timesheet (two screens
errors.def	intentionally invalid form - demonstrates

libonigwrap is a native dependency of TerminalGui and must remain in the same directory as the DataEntry executable.

Running the Compiler

dataentry	# opens a file-browser c
dataentry myform.def	# preview the form (or
dataentry myform.def --build	# generate + build with
dataentry myform.def --build --output ./out	# specify outpu

Flag	Meaning
------	---------

<i>(no flags)</i>	Load the file, preview valid forms or show errors
<code>--build</code>	Generate a VB.NET project, compile it, and publish a self-contained executable
<code>--output <dir></code>	Write generated files to <code><dir></code> (default: a subfolder named after the <code>.def</code> file, beside it)

Interactive preview mode

When the form has **no errors** the compiler renders it live so you can tab through fields, check positions, colours, and format hints before building. No data is written to disk during preview.

Key	Action
Tab / Enter	Move to next field
Shift-Tab	Move to previous field
Ctrl+S	Simulate a save (preview message only — no file written)
F1	Help dialog
F3	Clear fields
Page Up / Page Down	Switch screens in a multi-screen form
Shift-PgUp / Shift-PgDn	Previous / next record (preview message only)
Shift-Home / Shift-End	First / last record (preview message only)
F10	Compile & Build the form
File menu	Open a different <code>.def</code> file, Compile & Build, Quit

When the form has **errors** the compiler shows a scrollable list of every parse and validation problem with line numbers. Press **Q** or **Esc** to exit.

Build output

`--build` generates a complete [VB.NET](#) project, compiles it with `dotnet publish`, and produces a self-contained single-file executable:

```

<output-dir>\
  <name>.vbproj
  Program.vb
  MainForm.vb
  DataFile.vb
  FormatHelper.vb
  ColorHelper.vb
  ValidationFunctions.vb  (only when VALIDATE WITH or VALID
publish\
  <name>.exe              (Windows) or <name> (Linux/macOS
  libonigwrap.dll/.so     native dependency – must stay be

```

Run the generated application directly from the `publish\` subfolder:

```

publish\sample.exe      (Windows)
./publish/sample        (Linux/macOS)

```

Language Reference

A `.def` file has two required sections plus one optional section, in this order:

1. DATA-SECTION — describes the output file and its record layout.
2. SCREEN-SECTION — describes the data-entry screens.
3. VALIDATE-SECTION (*optional*) — named validation blocks with rules.

Lines beginning with `*` or `//` are comments.

DATA-SECTION

```

DATA-SECTION
  FILE <path> [APPEND|NOAPPEND] LRECL=<n> LEND=<mode>

```

FILE keyword

Keyword	Required	Meaning
FILE <path>	Yes	Path to the output data file (relative paths are relative to the generated app's working directory)
APPEND	No (default)	Add records to the end of an existing file; create if absent

NOAPPEND	No	Delete the existing file at startup and start fresh
LRECL=<n>	Yes	Logical record length in bytes — all records are exactly this wide
LEND= <mode>	No (default CRLF)	Line-ending written after each record

LEND modes:

Value	Written after each record
CRLF	Carriage-return + line-feed (\r\n) — standard Windows
LF	Line-feed only (\n) — standard Unix/Linux
CR	Carriage-return only (\r)
NONE	Nothing — records are written end-to-end

RECORD Definition

```
RECORD <name>
    <fieldname> [START=<n>] LEN=<n>
    FORMAT=<mask>.
    <fieldname> [START=<n>] LEN=<n>
    ...
```

- <name> — identifier used in INTO clauses on the screen.
- START=<n> — 1-based byte position of the first character of this field.
Optional on every field (see implicit positioning below).
- LEN=<n> — number of bytes this field occupies in the record.
- FORMAT=<mask> . — input mask (see below). The mask **must end with a dot**.

Implicit field positioning

START= is optional. When omitted the field starts immediately after the last byte of the preceding field (or at column 1 for the very first field).

```
RECORD CUST
    FIRSTNAME START=1 LEN=20 * explicit - starts at col 1
    LASTNAME   LEN=20   * implicit - starts at col 21
    CITY       LEN=20   * implicit - starts at col 41
```

Use an explicit `START=` on any field to leave **dead space** (filler bytes) in the record — useful when a layout reserves bytes for future use or must interoperate with an existing file format:

```

RECORD EMPLOYEE
    EMPNO    START=1    LEN=6    * cols 1-6
    NAME      START=8    LEN=30   * cols 8-37 (col 7 is dead s
    DEPT      START=11   LEN=4     * cols 38-41 (implicit, foll
    SALARY    START=50   LEN=10   * cols 50-59 (cols 42-49 are

```

Rules:

- Column numbers are 1-based — `START=1` is the first byte.
- `START=0` or any negative value is a hard error.
- If a field's range overlaps a previously-defined field, that is a hard error.
- If a field's last byte exceeds `LRECL`, that is a hard error.

FORMAT Mask Characters

Character	Meaning
X	Any character, stored as-is
U	Letter, forced to upper-case on store
L	Letter, forced to lower-case on store
9	Digit (0–9); space if blank
Z	Digit (0–9); zero-filled if blank
\c	Literal character c inserted automatically on store (e.g. \- inserts a hyphen)

Alignment when storing:

- Fields whose mask contains only 9 or Z placeholders (plus any \c literals) are **right-adjusted** (padded on the left with spaces).
- All other fields are **left-adjusted** (padded on the right with spaces).

Format hints:

When a mask contains embedded literals the compiler automatically places a muted grey hint label to the right of the field showing the expected input pattern:

Mask	Hint shown	Notes
999\ - 999\ - 9999	###-###-####	Type 10 digits; hyphens inserted on save
99\ / 99\ / 9999	##/##/####	Type 8 digits; slashes inserted on save

ZZ.99	##.##	Type 4 digits; dot inserted on save
999999	(none)	Pure digits — self-explanatory

In the hint, # represents a digit position, @ represents any character, and ^ represents a letter.

Mask examples:

Mask	LEN	Input typed	Stored value
XXX	3	Hi	Hi
999	3	42	42
ZZZ	3	42	042
UU	2	tx	TX
999\ -9999	8	5551234	555-1234
99\ /99\ /9999	10	12252025	12/25/2025

SCREEN-SECTION

A form may have **one or more screens**. In a multi-screen form the user presses **Page Down** to advance and **Page Up** to go back.

SCREEN-SECTION

```

SCREEN <name>  [COLOR=<fg>0n<bg>]  [FG=<color>]  [BG=<color>]
    PROMPT "<text>"  ROW=<n>  COL=<n>  [COLOR=<color>]
    FIELD  ROW=<n>  COL=<n>  LEN=<n>  INTO <record>.<field>
        [VALIDATE WITH <function>]  [PROTECTED]
        [NORMAL=<color>  FOCUS=<color>  ERROR=<color>  FULL=<A
...

SCREEN <name2>  ...

```

SCREEN line

Keyword	Meaning
<name>	Identifier for this screen
COLOR=<fg>0n<bg>	Default foreground and background for the entire screen
FG=<color>	Default foreground colour only

BG=<color> Default background colour only

PROMPT / LABEL

Places a static text label on the screen. Used for titles, field labels, box-drawing borders, and hints.

```
PROMPT "<text>" ROW=<n> COL=<n>
      COLOR=<color>
```

- LABEL is an alias for PROMPT.
- The COLOR= line is optional; if omitted the screen default colour is used.
- Box-drawing characters (┌ ┐ └ ┘ │ ┌ ┐ └ ┘ ┌ ┐ └ ┘) work directly in quoted strings for panel borders and title banners.

FIELD line

Keyword	Required	Meaning
ROW=<n>	Yes	Screen row (1-based)
COL=<n>	Yes	Screen column (1-based)
LEN=<n>	Yes	Width of the input box in characters — must match the FORMAT mask length
INTO <rec>. <fld>	Yes	Which record field receives this value on save
VALIDATE WITH <fn>	No	Validation function called when the field loses focus
PROTECTED	No	Display-only field — user can see but not edit (see below)

An optional inline label can precede the position keywords:

```
FIELD "Phone" ROW=8 COL=2 LEN=12 INTO CUST.CPHONE
```

PROTECTED fields

Adding PROTECTED to a FIELD line makes it **display-only**:

```
FIELD ROW=5 COL=30 LEN=10 INTO TIME.GROSS PROTECTED
```

- The field is rendered on screen but the user cannot tab into it or type in it (3270-style protected field semantics).
- Its value **is** saved to and loaded from the data file normally.
- Typically used for calculated fields whose values are set by a **VALIDATE-SECTION** assignment rule (see below). If a **VALIDATE-SECTION** rule targets a screen field that is *not* **PROTECTED**, the compiler emits a warning because the user could overwrite the calculated value.

Field colour states and behaviour

On the line(s) following a **FIELD**, specify colour states and full-field behaviour:

NORMAL=<color> **FOCUS**=<color> **ERROR**=<color> **FULL**=<ADVANC

Attribute	When it applies
NORMAL =<color>	Field is visible but not focused
FOCUS =<color>	Field is currently active
ERROR =<color>	Validation returned an error
FULL =ADVANCE	When the field fills, automatically move to the next field (or save if last). This is the default.
FULL =STAY	When the field fills, hold the cursor and wait for Tab or Enter. Useful for fixed codes where the operator should review before advancing.

All colour attributes accept either a bare colour name (**White**) or the <fg>On<bg> shorthand (**WhiteOnBlue**).

Colour Names

The 16 standard console colours:

Black, DarkRed (or Red), DarkGreen (or Green), DarkYellow (or Yellow), DarkBlue (or Blue), DarkMagenta (or Magenta), DarkCyan (or Cyan), Gray, DarkGray, BrightRed, BrightGreen, BrightYellow, BrightBlue, BrightMagenta, BrightCyan, White

Built-in defaults (when not specified in the DSL):

Element	Default
Screen background	Gray on DarkBlue
Field normal	Inherits screen default
Field focus	Black on Cyan
Field error	White on DarkRed

VALIDATE WITH

When a field has `VALIDATE WITH <function>`, that function is called every time the field loses focus (the user tabs away). It receives the field value as a string and must return:

Return value	Meaning
True	Accept the value
False	Reject; show the field in ERROR colour and display a status message
A string	Replace the field value with this string, then accept

If a matching `VALIDATE <function>` block exists in the `VALIDATE-SECTION` the compiler generates a **real implementation** in `ValidationFunctions.vb`. If no block exists, the compiler generates a **stub** — a TODO placeholder for you to fill in manually.

VALIDATE-SECTION

The optional `VALIDATE-SECTION` lets you define validation logic directly in the `.def` file. Each named block corresponds to a `VALIDATE WITH` reference on a screen field.

VALIDATE-SECTION

```
VALIDATE <name>
    NOT EMPTY [MESSAGE "<text>"]
    VALUE IS BETWEEN <lo> AND <hi> [MESSAGE "<text>"]
    <targetfield> IS <expression> [MESSAGE "<text>"]
    ...

VALIDATE <name2>
    ...
```

A single **VALIDATE** block may contain **any number** of rules; they are evaluated in order when the field loses focus. The first rule that fails stops evaluation and displays the message (or a default message if **MESSAGE** is omitted).

Rule kinds

NOT EMPTY

```
NOT EMPTY
NOT EMPTY MESSAGE "A value is required."
```

Fails if the field value is blank or all spaces.

VALUE IS BETWEEN

```
VALUE IS BETWEEN 1 AND 168
VALUE IS BETWEEN 0.5 AND 999.99 MESSAGE "Hours must be between
```

Fails if the field value, parsed as a number, is outside the inclusive range [lo, hi]. Also fails if the value is not a valid number. Decimal bounds are supported (0.5, 999.99).

Assignment rule (<target> IS <expression>)

```
GROSS IS HOURS * RATE
TAX    IS GROSS * 0.2
TOTAL IS SUBTOTAL + TAX
```

Computes an arithmetic expression using field values and stores the result in <target>. This rule **does not fail** — it is an unconditional assignment that fires when the validated field loses focus.

- <target> must be a field name defined in a **RECORD**.
- Operands can be field names or numeric literals.
- Operators: +, -, *, /.
- Operator precedence is **left-to-right** (no parentheses).
- If <target> appears on the screen it should be declared **PROTECTED** to prevent the user from overwriting the calculated value.

Status bar

The generated application shows validation messages in a **status bar** at the bottom of the screen. When a rule fails its **MESSAGE** text (or a default message) is displayed

there. The bar clears when the field passes validation.

Example

VALIDATE-SECTION

VALIDATE CheckHours

NOT EMPTY MESSAGE "Hours worked is required."

VALUE IS BETWEEN 0.01 AND 168 MESSAGE "Hours must be betwe

VALIDATE CalcGross

NOT EMPTY MESSAGE "Rate is required."

GROSS IS HOURS * RATE

Paired with the screen fields:

```
FIELD "Hours" ROW=5 COL=10 LEN=6 INTO TIME.HOURS  VALIDATE WIT
FIELD "Rate"  ROW=6 COL=10 LEN=8 INTO TIME.RATE    VALIDATE WIT
FIELD                ROW=7 COL=10 LEN=10 INTO TIME.GROSS  PROTECTED
```

When the operator tabs away from **Rate**, the CalcGross block fires: first it checks that RATE is not empty, then it computes $GROSS = HOURS * RATE$ and writes the result into the protected GROSS field on screen and into the record buffer.

Runtime Hotkeys (Generated Application)

Every application produced by the compiler has these built-in keys.

Key	Action
Tab / Enter	Move to next field
Shift-Tab	Move to previous field
Home	Go to start of current field
End	Go to end of current field
Insert	Toggle insert / overwrite mode
Backspace	Delete character to the left
Delete	Delete character under cursor
Last field full / Enter	Automatically saves the record and clears fields for next entry

Ctrl+S	Save record manually and clear fields
F3	Clear all fields (cancel current entry)
F10	Quit the application
Page Up / Page Down	Switch between screens (multi-screen forms)
Shift-PgUp	Load previous record for editing
Shift-PgDn	Load next record for editing
Shift-Home	Load the first record
Shift-End	Load the last record

Worked Example

1 — Write the definition file (**cust.def**)

DATA-SECTION

FILE customers.dat APPEND LRECL=87 LEND=CRLF

RECORD CUST

CNAME START=1 LEN=30
FORMAT=XXXXXXXXXXXXXXXXXXXXXXXXXXXXX.
CADDR LEN=30
FORMAT=XXXXXXXXXXXXXXXXXXXXXXXXXXXXX.
CSTATE LEN=2
FORMAT=UU.
CZIP LEN=5
FORMAT=99999.
CPHONE LEN=12
FORMAT=999\ -999\ -9999.
CEMAIL LEN=8
FORMAT=XXXXXXX.

SCREEN-SECTION

SCREEN CUST-ENTRY COLOR=WhiteOnBlue

PROMPT "┌───────────────────────────────────┐" ROW=1 COL=5
COLOR=WhiteOnBlue
PROMPT "┆ CUSTOMER ENTRY ┆" ROW=2 COL=5
COLOR=BrightYellowOnBlue
PROMPT "└───────────────────────────────────┘" ROW=3 COL=5
COLOR=WhiteOnBlue
PROMPT "Name : " ROW=5 COL=5
FIELD ROW=5 COL=14 LEN=30 INTO CUST.CNAME
NORMAL=WhiteOnBlue FOCUS=BlackOnCyan ERROR=WhiteOnRed
PROMPT "Address: " ROW=6 COL=5
FIELD ROW=6 COL=14 LEN=30 INTO CUST.CADDR
NORMAL=WhiteOnBlue FOCUS=BlackOnCyan ERROR=WhiteOnRed
PROMPT "State : " ROW=7 COL=5
FIELD ROW=7 COL=14 LEN=2 INTO CUST.CSTATE
NORMAL=WhiteOnBlue FOCUS=BlackOnCyan ERROR=WhiteOnRed
PROMPT "Zip : " ROW=7 COL=19
FIELD ROW=7 COL=28 LEN=5 INTO CUST.CZIP
NORMAL=WhiteOnBlue FOCUS=BlackOnCyan ERROR=WhiteOnRed
PROMPT "Phone : " ROW=8 COL=5
FIELD ROW=8 COL=14 LEN=12 INTO CUST.CPHONE
NORMAL=WhiteOnBlue FOCUS=BlackOnCyan ERROR=WhiteOnRed
PROMPT "Email : " ROW=9 COL=5
FIELD ROW=9 COL=14 LEN=8 INTO CUST.CEMAIL
NORMAL=WhiteOnBlue FOCUS=BlackOnCyan ERROR=WhiteOnRed

Notes on this form:

- CNAME has START=1; all subsequent fields omit START= and are positioned implicitly — CADDR at col 31, CSTATE at col 61, etc.

- CPHONE has a `FORMAT=999\~999\~9999.` mask. The compiler automatically shows a `###-###-####` hint to the right of the field. Type 10 digits; the hyphens are inserted automatically when the record is saved.
- CSTATE uses `FULL=STAY` so the cursor waits after the 2nd character.
- `LRECL=87 = 30+30+2+5+12+8 = 87` bytes per record.

2 — Preview the form

```
dataentry cust.def
```

The compiler opens a TerminalGui window. Tab through the fields to verify placement. Notice the grey `###-###-####` hint beside the phone field. Press **Esc** (File → Quit) when done.

3 — Compile and build

```
dataentry cust.def --build
```

The compiler generates the [VB.NET](#) project in `cust\` beside `cust.def`, compiles it, and publishes a self-contained executable:

```
cust\publish\cust.exe      (Windows)
cust\publish\cust          (Linux/macOS)
cust\publish\libonigwrap.dll (native dependency – keep beside
```

4 — Run the generated application

```
cust\publish\cust.exe      (Windows)
./cust/publish/cust        (Linux/macOS)
```

Enter customer records. The phone field shows `###-###-####` — type 10 digits and the saved record will contain 315-617-6379. Completing the last field (or pressing **Ctrl+S**) saves each record and clears the form for the next entry. Press **F10** to quit.

Records are written to `customers.dat` as 87-byte fixed-length lines.

Error Messages

Message	Cause	Fix
---------	-------	-----

DATA-SECTION is missing a FILE path	No FILE keyword found	Add FILE <path> LRECL=<n> after DATA-SECTION
LRECL must be greater than zero	LRECL=0 or omitted	Specify LRECL=<n> with a positive integer
Duplicate record name '<name>'	Two RECORD blocks share a name	Rename one
Duplicate field name '<name>'	Two fields in the same record share a name	Rename one
START=<n> is invalid	START=0 or a negative value was specified	Column numbers are 1-based; use START=1 for the first column
Field '<f>' overlaps a previously defined field	Two fields occupy the same byte range	Adjust START= or LEN= so the ranges do not overlap
Field '<f>' exceeds LRECL	START + LEN exceeds the record length	Reduce LEN, adjust START, or increase LRECL
FORMAT mask is longer than LEN	Mask has more positions than LEN — data would be truncated	Increase LEN to match the mask length (count each \c pair as one position)
FORMAT mask is shorter than LEN (<i>warning</i>)	Mask has fewer positions than LEN	Acceptable for partial-fill fields; increase mask length to fill completely
Unknown record '<rec>'	Screen field INTO references an undefined record	Check record name spelling
Unknown field '<rec>.<fld>'	Screen field INTO references an undefined field	Check field name spelling

FILE path contains '..' or is absolute (<i>warning</i>)	Path may write outside the working directory	Verify the path is intentional
Duplicate VALIDATE block name '<name>'	Two VALIDATE blocks share a name	Rename one
VALIDATE '<blk>': target field '<f>' is not defined in any RECORD	Assignment rule targets an unknown field	Check the field name spelling or add the field to a RECORD
VALIDATE '<blk>': expression references unknown field '<f>'	Expression operand is not a record field	Check the field name spelling
VALIDATE '<blk>': target field '<f>' is on the screen but not PROTECTED (<i>warning</i>)	Calculated field is editable — user can overwrite computed value	Add PROTECTED to that screen FIELD
VALIDATE WITH '<fn>' – ensure this function is defined (<i>warning</i>)	No matching VALIDATE block found in VALIDATE-SECTION	Add a VALIDATE <fn> block, or implement the stub in ValidationFunctions.vb